

# Средства симуляции ЦП и ОС

Лекция №6

Державин Андрей  
Шурыгин Антон

# Автоматизация в симуляторах

# Автоматизация СК

- Удобно ли было писать дублирующиеся куски кода?
- А отписывать декодер?
- Или может синхронизировать декодер и исполнение?



```
47  */
48  void virtual execute(sim::isa::Instruction insn) {
49      switch (insn.opc) {
50          case sim::isa::Opcode::kAdd: {
51              auto res = cpu.getReg(insn.src1) + cpu.getReg(insn.src2);
52              cpu.setReg(insn.dst, res);
53              cpu.pc += 1;
54              break;
55          }
56          case sim::isa::Opcode::kHalt: {
57              cpu.finished = true;
58              break;
59          }
60          case sim::isa::Opcode::kJump: {
61              cpu.pc = cpu.getReg(insn.dst);
62              break;
63          }
64          case sim::isa::Opcode::kLoad: {
65              auto data = cpu.memory->load(cpu.getReg(insn.src1));
66              cpu.setReg(insn.dst, data);
67              cpu.pc += 1;
68              break;
69          }
70          case sim::isa::Opcode::kStore: {
71              auto data = cpu.getReg(insn.dst);
72              cpu.memory->store(cpu.getReg(insn.src1), data);
73              cpu.pc += 1;
74              break;
75          }
76          case sim::isa::Opcode::kBeq: {
77              bool eq = cpu.getReg(insn.src1) == cpu.getReg(insn.src2);
78              cpu.pc = eq ? cpu.getReg(insn.dst) : cpu.pc + 1;
79              break;
80          }
81          default:
82              assert(false && "Unknown instruction");
83      }
84  }
```

# Автоматизация СК

- Спецификации ISA описывают поведение процессора
- Полная исполняемая спецификация – полезное подспорье для архитекторов при разработке или расширении СК

*Jump unconditional*

"JP B*i* + K" ( $:= fmi = 02$ )  $\rightarrow (P \leftarrow B[i] + K; p \leftarrow 3)$ ; *jump*

*Jump on X[j] conditions*

"ZR Xj K" ( $:= fmi = 030$ )  $\rightarrow ((X[j] = 0) \rightarrow (P \leftarrow K; p \leftarrow 3))$ ; *zero*

"NZ Xj K" ( $:= fmi = 031$ )  $\rightarrow ((X[j] \neq 0) \rightarrow (P \leftarrow K; p \leftarrow 3))$ ; *non zero*

"PL Xj K" ( $:= fmi = 032$ )  $\rightarrow ((X[j] \geq 0) \rightarrow (P \leftarrow K; p \leftarrow 3))$ ; *plus or position*

"NG Xj K" ( $:= fmi = 033$ )  $\rightarrow ((X[j] < 0) \rightarrow (P \leftarrow K; p \leftarrow 3))$ ; *negative*

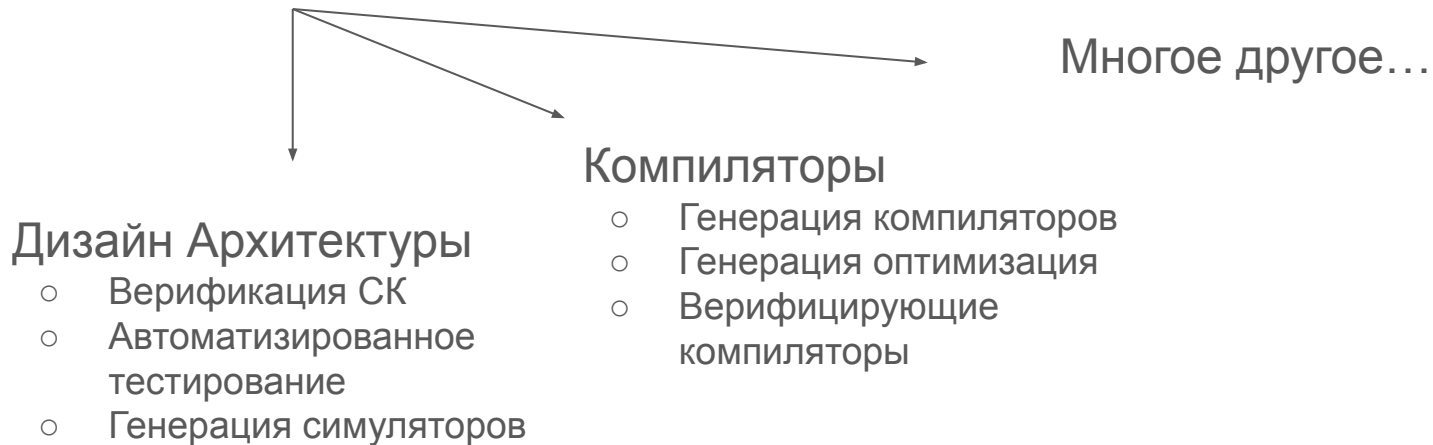
Фрагмент спецификации CDC 6600

## Table of Contents

|         |    |
|---------|----|
| J       | 3  |
| SYSCALL | 4  |
| STP     | 5  |
| RORI    | 6  |
| SLTI    | 7  |
| ST      | 8  |
| BDEP    | 9  |
| CLS     | 10 |
| ADD     | 11 |
| BNE     | 12 |
| BEQ     | 13 |
| LD      | 14 |
| AND     | 15 |
| SSAT    | 16 |

# Автоматизация СК

- Если есть машиночитаемая спецификация – есть всё?



# Simics

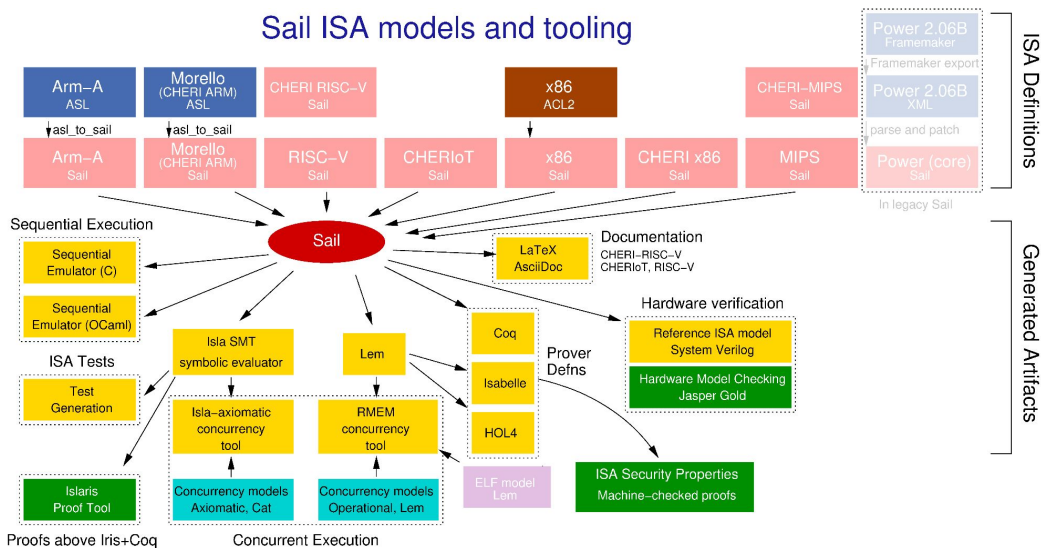
- Фреймворк для полносистемной симуляции + JIT
- Имеет самописные:
  - ADL для ISA
  - SDL для девайсов (DML)
- Разрабатывается Intel
- Что думаете насчет описания?

```
instruction ADD_L({REG}, {REG_OR_MEM})
  pattern
    op_h2 == 0 && opl == 0 && d == 1
    && opm == 0
  syntax
    "add {REG},{REG_OR_MEM}"
  semantics
    # {
      ireg_t op1 = {REG};
      ireg_t op2 = {REG_OR_MEM};
      ireg_t dst = op1 + op2;
      EFLAGS_ADD(dst, op1, op2, w, os);
      SET({REG_W}, dst);
    # }
  attributes
    type = IT_ALU
```

Семантика инструкции x86-64 add-to-left: Simics

# SAIL

- Sail — это язык для определения семантики архитектуры набора инструкций процессоров: архитектурной спецификации поведения машинных инструкций



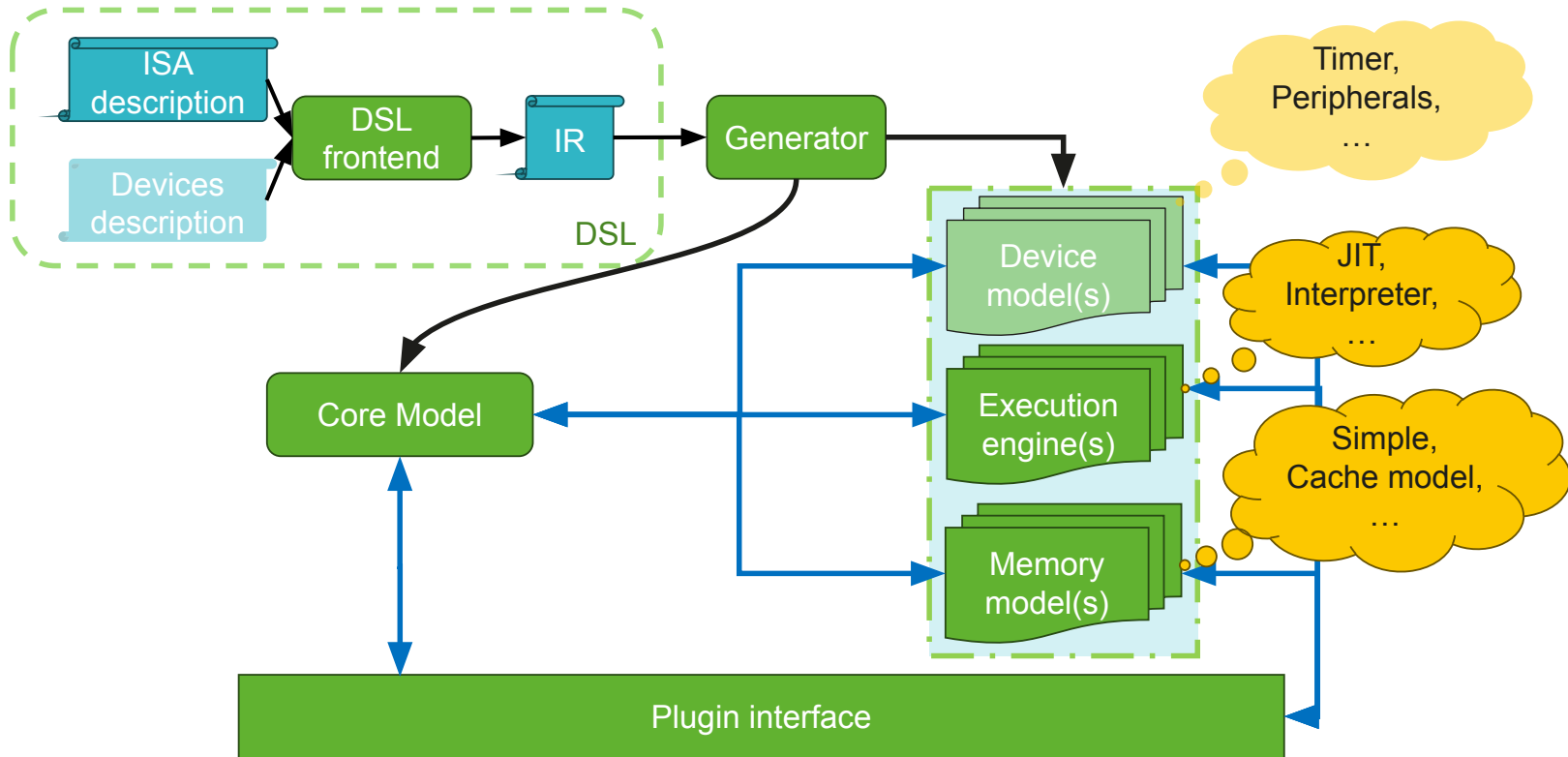
# SAIL

- Рассмотрим инструкцию сложения
  - Вычисляемая во время исполнения ширина битовых типов данных
  - Использование императивно обновляемых локальных переменных
- Что думаете насчет данного описания СК?

```
function integer_arithmetic_addsub_carry
  (d, datasize, m, n, setflags, sub_op) = {
    result : bits('datasize') = undefined : bits('datasize');
    let operand1 : bits('datasize') = X(n);
    operand2 : bits('datasize') = X(m);
    nzcv : bits(4) = undefined : bits(4);
    if sub_op then {
      operand2 = ~(operand2)
    };
    (result, nzcv) = AddWithCarry(operand1, operand2, PSTATE.C);
    if setflags then {
      (PSTATE.N @ PSTATE.Z @ PSTATE.C @ PSTATE.V) = nzcv
    };
    X(d) = result
  }
}
```

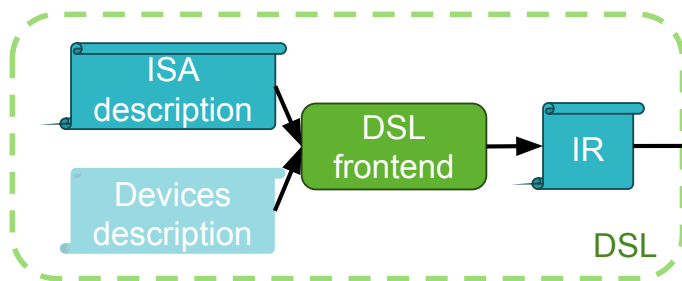


# Архитектура DSL



# Co-design

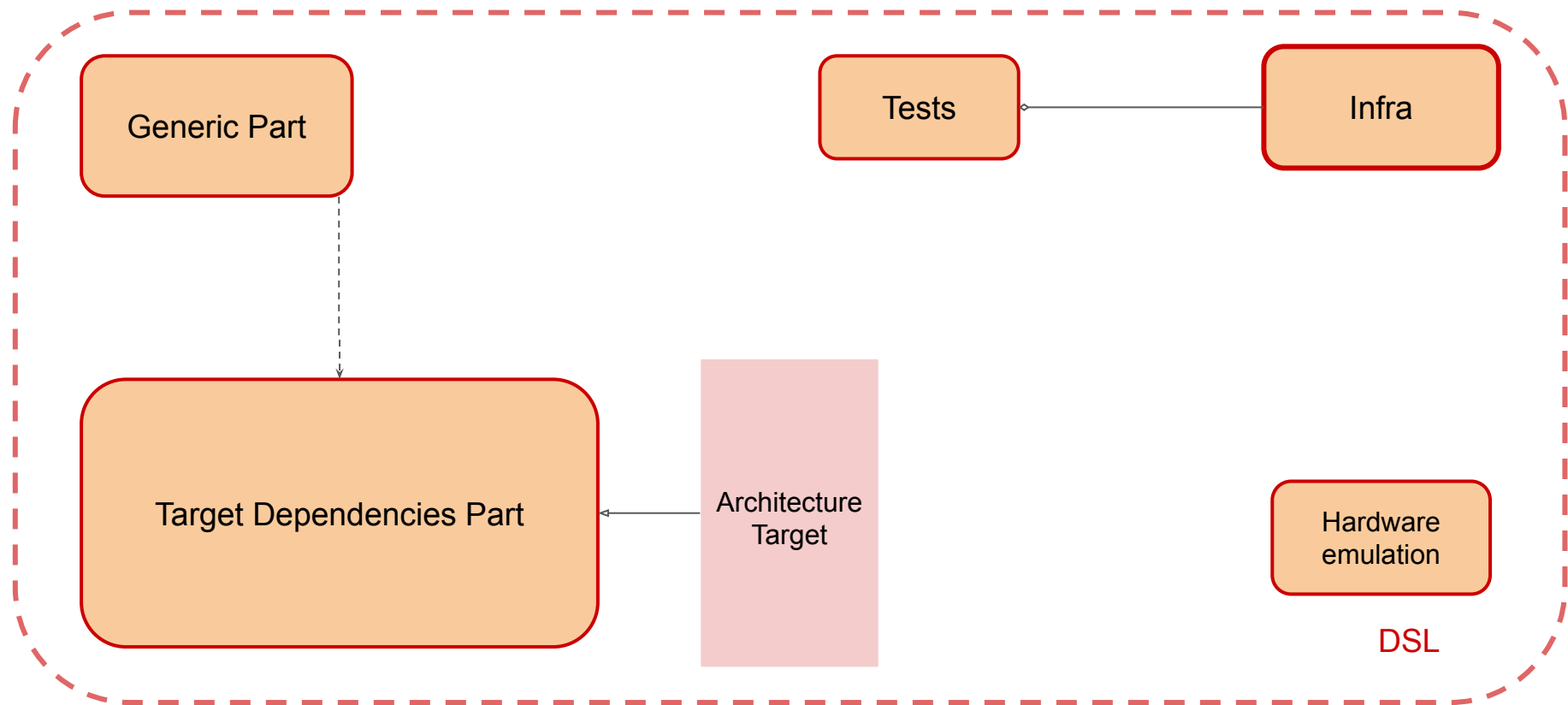
- Если вкратце, наша задача – совместное проектирование



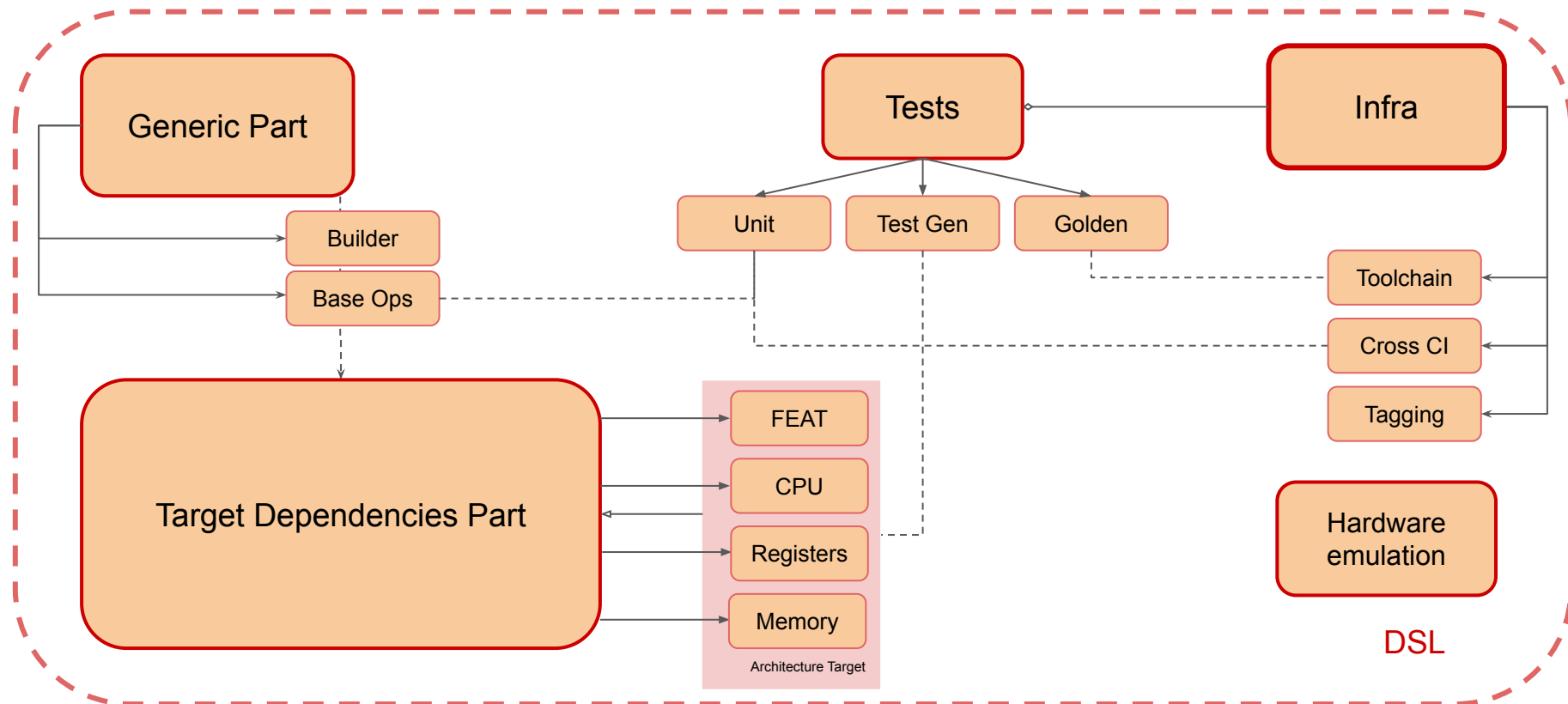
Generator



# Архитектура DSL



# Архитектура DSL



# DSL на Ruby

- Давайте изобретём эффективное описание архитектуры СК?
- Разбираем ADD из RV32I

```
module RV32I
  Instruction(:add) {
    encoding {
      field(:rd, 11, 7, :reg), field(:rs1, 19, 15, :reg),
      field(:rs2, 24, 20, :reg), field(:opcode, 6, 0, 0110011),
      field(:funct7, 31, 25, 0000000), field(:funct3, 14, 12, 000) }
    asm { ADD }
    code { rd[] = rs1 + rs2 }
  }
end
```

# DSL на Ruby

- Добавим немного сахара
- Однако описание получилось немного бедным, нет?

```
module SimInfra
  def self.format_r(opcode, funct3, funct7)
    return :R, [
      field(:rd, 11, 7, :reg),
      field(:rs1, 0, 0, :reg),
      field(:rs2, 0, 0, :reg),
      field(:opcode, 0, 0, opcode),
      field(:funct7, 0, 0, funct7),
      field(:funct3, 0, 0, funct3),
    ]
  end

  def self.format_r_alu(name)
    funct3, funct7 =
    {
      add: [0,0],
      sub: [0, 1 << 5]
    }[name]
    format_r 0, funct3, funct7
  end
end
```

```
module RV32I
  Instruction(:add) {
    encoding *format_r_alu(:add)
    asm { ADD }
    code { rd[] = rs1 + rs2 }
  }
end
```

# DSL на Ruby

- Укажем явно входные аргументы инструкции

```
class XReg
  attr_reader :name
  def initialize(name)
    @name = name
  end

  def to_s
    @name.to_s
  end
end

def XReg(name); XReg.new(name); end
```

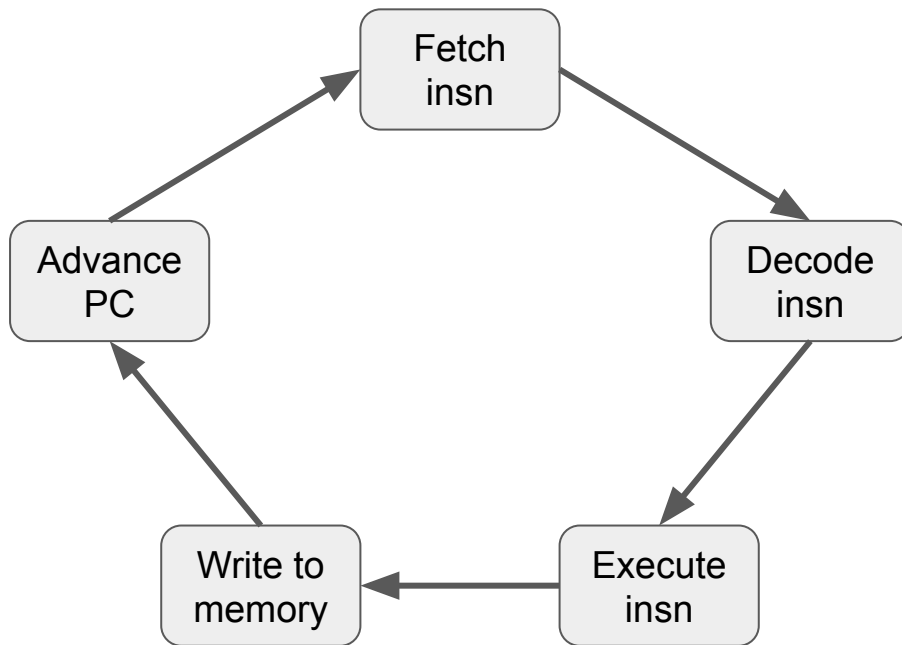
```
module RV32I
  Instruction(:ADD, XReg(:rd), XReg(:rs1), XReg(:rs2)) {
    encoding *format_r_alu(:add, rd, rs1, rs2)
    asm { "ADD #{rd}, #{rs1}, #{rs2}" }
    code { rd[] = rs1 + rs2 }
  }
end
```



Декодирование

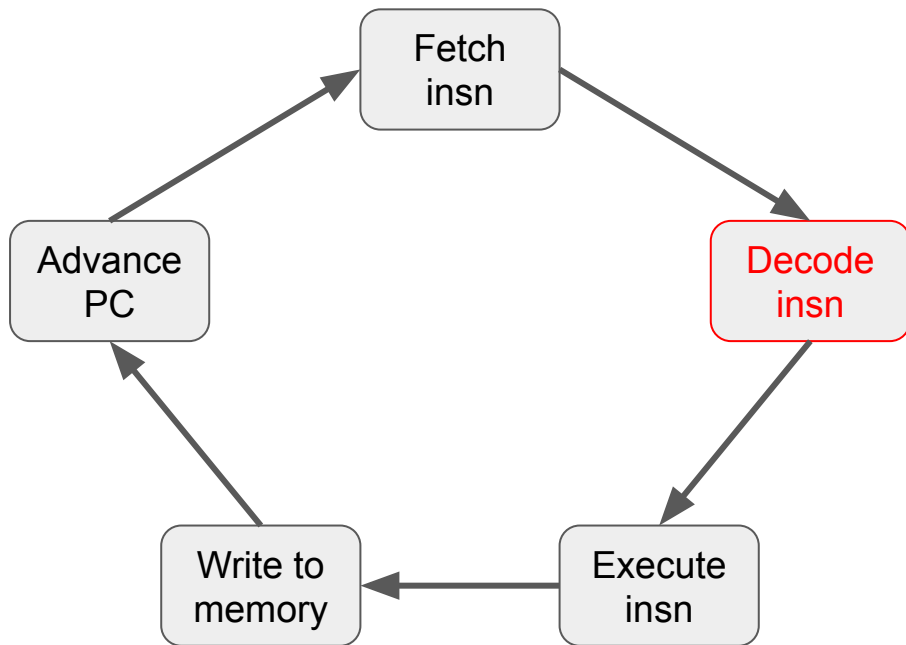
# Стадии интерпретатора

- 5 стадий интерпретации



# Стадии интерпретатора

- 5 стадий интерпретации



# Анализ выражений

**letter** = [a-zA-Z]  
**digit** = [0-9]  
**digits** = digit digit\*

**identifier** = letter | (letter | digit | \_)\*

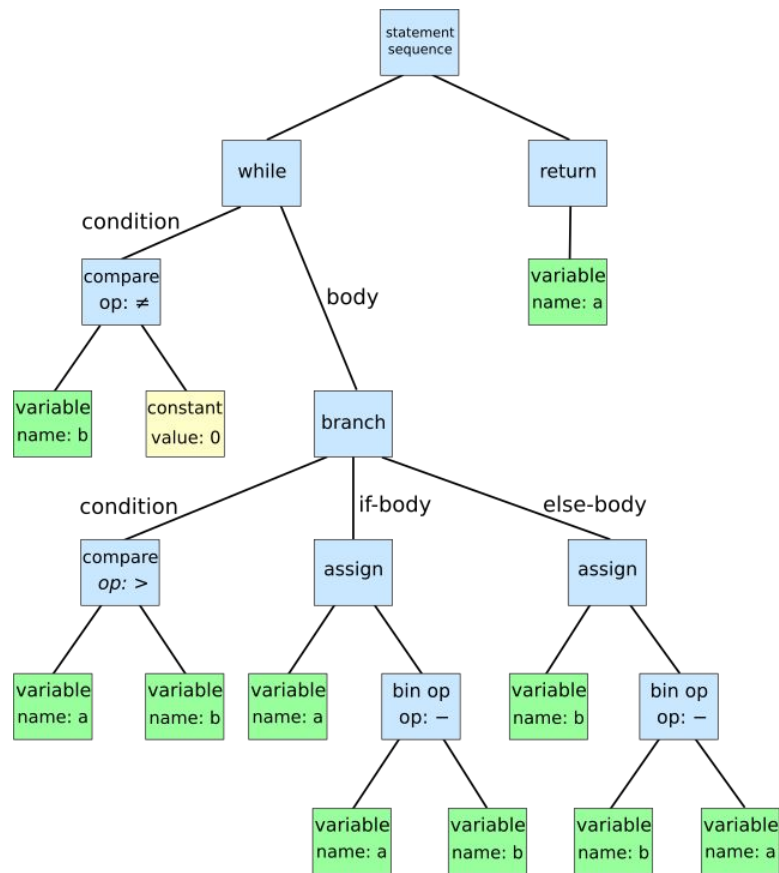
**fraction** = .digits |  $\epsilon$

**exponent** = ((E | e) (+ | - |  $\epsilon$ )) digits) |  $\epsilon$

**number** = digits fraction exponent

**operator** = + | - | \* | / | > | >= | < | <= | = | ==

**parenthesis** = ( | )



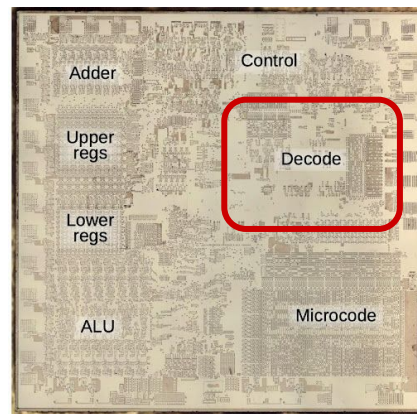
# Декодирование как анализ

- Машинное представление инструкций – тоже всего лишь язык
- Особенности языка, позволяющие строить декодеры оптимально:
  - Длина инструкций
  - Префиксный код
  - Режим процессора



# Процедура декодирования

- В реальном процессоре декодер – это блок из логических элементов
- В функциональном симуляторе – процедура на языке программирования



Intel 8086 die

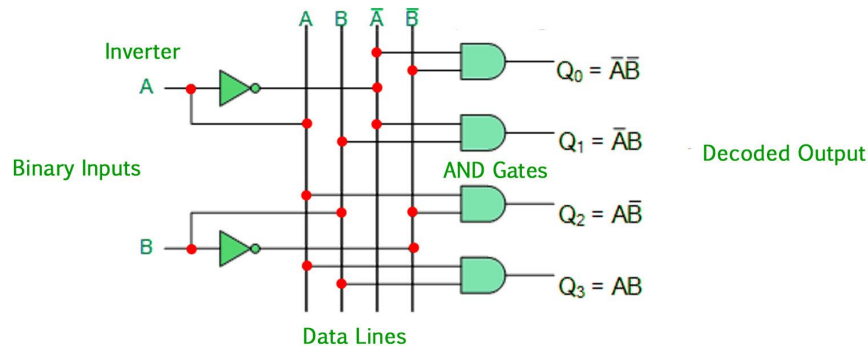
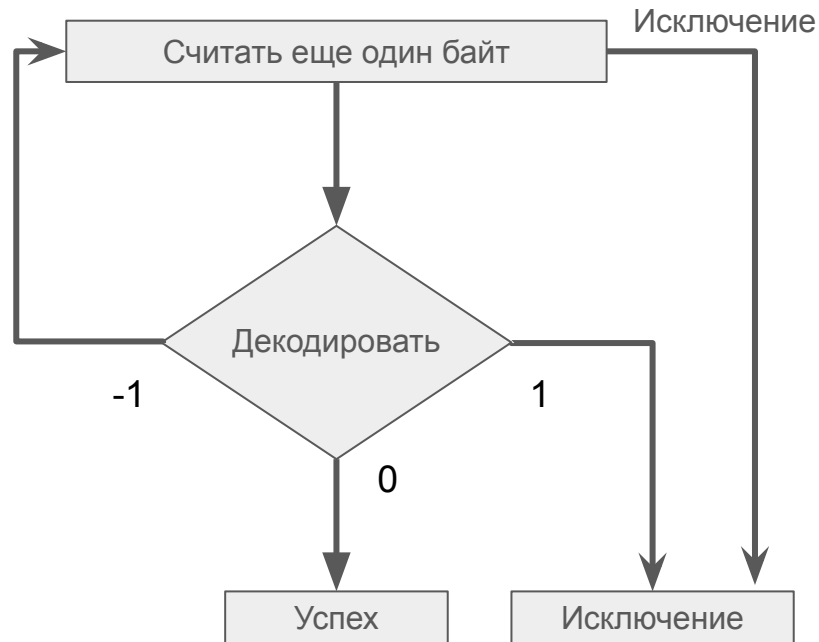
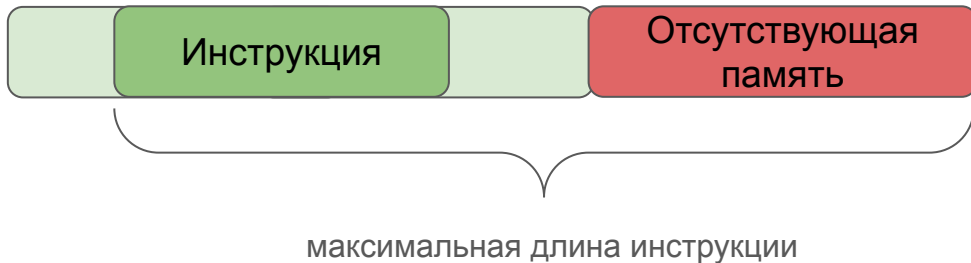


Схема двоичного декодера 2-4

# Процедура декодирования

- Разберём декодер для инструкций с переменной длиной
- А это точно лучшее решение?
- Предложение: будем читать с запасом



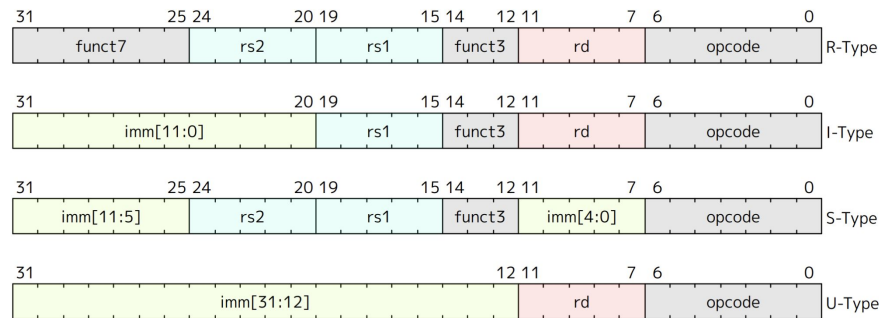
Блок-схема процедуры декодирования для машинного слова переменной длины

# Оптимизация декодирования



# Распознавание шаблонов

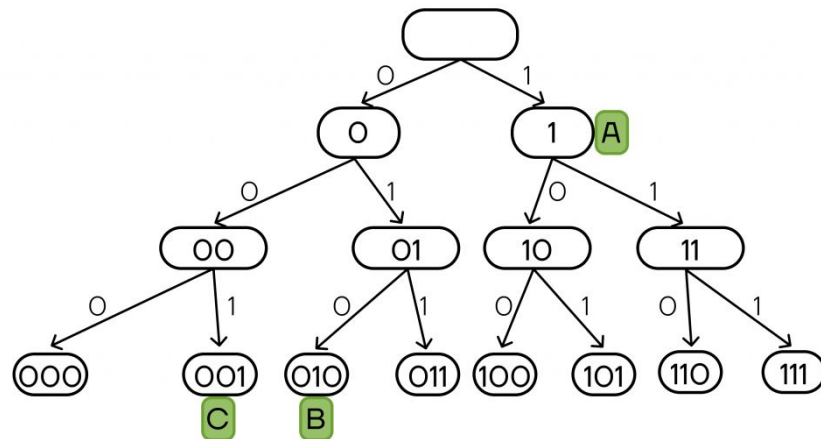
- Сопоставление массива байт набору шаблонов из документации архитектуры
- Любой входной последовательности бит соответствует минимум один шаблон



Основные типы кодировок RV32I

# Оптимизация декодирования

- Число шаблонов для сложной архитектуры может быть большим
- Переход от линейного декодера к дереву поиска

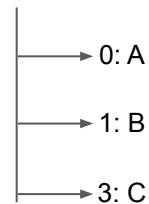


# Дерево декодирования

- Построим своё дерево
- Для простоты ограничимся тремя шаблонами: A, B, C

|    |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|
|    | 7 |   |   |   |   |   | 0 |
| A: | 1 | 0 | 0 | 1 | x | x | x |
| B: | 1 | 1 | 1 | x | x | x | x |
| C: | x | 0 | 1 | x | x | x | x |

[6:5]

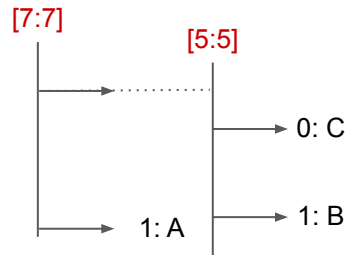


# Дерево декодирования

- Построим своё дерево
- Для простоты ограничимся тремя шаблонами: A, B, C
- А что если не все так однозначно?

|    | 7 |   |   |   |   |   |   | 0 |
|----|---|---|---|---|---|---|---|---|
| A: | 1 | 0 | x | 1 | x | x | x | x |
| B: | 0 | 1 | 1 | x | x | x | x | x |
| C: | 0 | x | 0 | x | x | x | x | x |

|    |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|
| B: | 0 | 1 | 1 | x | x | x | x | x |
| C: | 0 | x | 0 | x | x | x | x | x |



# Алгоритм

```
MAKE_HEAD(instructions):
    lead_bits = GET_LEAD_BITS(instructions)
    maj_range = GET_MAJ_RANGE(lead_bits)
    msb, lsb = maj_range[-1], maj_range[0]
    width = msb - lsb + 1

    tree["range"] = (msb, lsb)
    tree["nodes"] = {}

    FOR node_value FROM 0 TO (1 << width) - 1:
        actual_node = node_value << lsb
        subtree = {}

        is_leaf, result = MAKE_CHILD(actual_node,
                                     (1 << width) - 1 << lsb,
                                     instructions,
                                     subtree)

        IF is_leaf == True:
            tree["nodes"][node_value] = result
        ELSE IF subtree ≠ {}:
            tree["nodes"][node_value] = subtree

    RETURN tree
```

Инициализация, создание корня

```
MAKE_CHILD(node, separ_mask, instructions, current_subtree):
    sublist = FILTER_INSTRUCTIONS(instructions, node, separ_mask)

    IF sublist empty:
        RETURN None, None

    IF len(sublist) == 1:
        RETURN True, sublist

    lead_bits = GET_LEAD_BITS(sublist, separ_mask)
    maj_range = GET_MAJ_RANGE(lead_bits)
    msb, lsb = maj_range[-1], maj_range[0]
    width = msb - lsb + 1

    current_subtree["range"] = (msb, lsb)
    current_subtree["nodes"] = {}

    new_mask = separ_mask | ((1 << width) - 1 << lsb)

    FOR node_value FROM 0 TO (1 << width) - 1:
        actual_node = node | (node_value << lsb)
        child_subtree = {}

        is_leaf, result = MAKE_CHILD(actual_node, new_mask,
                                     sublist, child_subtree)

        IF is_leaf == True:
            current_subtree["nodes"][node_value] = result
        ELSE IF child_subtree ≠ {}:
            current_subtree["nodes"][node_value] = child_subtree

    RETURN None, None
```

Шаг рекурсии

Декодеры в симуляторах

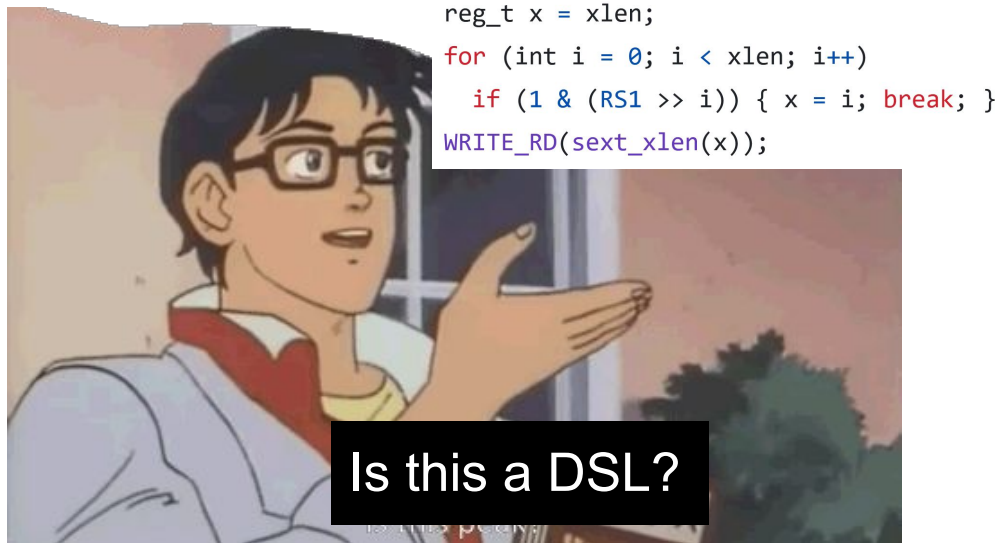
# QEMU

- Декодер генерируется автоматически
- Кодировки отписаны в .decode файлах
- Используется дерево декодирования через switch case
- Реализация заточена на – ~~нормальные~~ СК с инструкциями фиксированной длины
- Для x86 декодер отписан вручную

```
125 mnret      0111000  00010 00000 000 00000 1110011
126
127 # *** RV32I Base Instruction Set ***
128 lui        ..... 0110111 @u
129 {
130   lpad      label:20      00000 0010111
131   auipc     ..... 0010111 @u
132 }
133 jal         ..... 1101111 @j
134 jalr        ..... 000 ..... 1100111 @i
135 beq         ..... 000 ..... 1100011 @b
136 bne         ..... 001 ..... 1100011 @b
137 blt         ..... 100 ..... 1100011 @b
138 bge         ..... 101 ..... 1100011 @b
139 bltu        ..... 110 ..... 1100011 @b
140 bgeu        ..... 111 ..... 1100011 @b
141 lb          ..... 000 ..... 0000011 @i
142 lh          ..... 001 ..... 0000011 @i
143 lw          ..... 010 ..... 0000011 @i
144 lbu         ..... 100 ..... 0000011 @i
145 lhu         ..... 101 ..... 0000011 @i
146 sb          ..... 000 ..... 0100011 @s
147 sh          ..... 001 ..... 0100011 @s
148 sw          ..... 010 ..... 0100011 @s
149 addi        ..... 000 ..... 0010011 @i
150 slti        ..... 010 ..... 0010011 @i
151 sltiu       ..... 011 ..... 0010011 @i
152 xori        ..... 100 ..... 0010011 @i
153 #.cbo.prefetch_{i,r,m} instructions are ori with rd=x0 and not decoded.
154 ori         ..... 110 ..... 0010011 @i
155 andi        ..... 111 ..... 0010011 @i
156 slli        00000. .... 001 ..... 0010011 @sh
157
```

# SPIKE

- Рефересный функциональный симулятор RISC-V
- Заточен на расширяемость
- Макросы наше все
- Декодер - линейный
  - Но хоть с кэшированием



```
// fall back to linear search
auto matching = [insn_bits = insn.bits()](const insn_desc_t &d) {
    return (insn_bits & d.mask) == d.match;
};
auto p = std::find_if(custom_instructions.begin(),
                    custom_instructions.end(), matching);
if (p == custom_instructions.end()) {
    p = std::find_if(instructions.begin(), instructions.end(), matching);
    assert(p != instructions.end());
}
```



# gem5

- DSL присутствует
- Отписывает не только кодировку, но и семантику
  - И правда DSL!
- Генерирует дерево декодирования
- Не самый лучший пример...

```
void
exitSimulationLoopNow(uint64_t type_id,
    std::map<std::string, std::string> payload)
{
    new GlobalSimLoopExitEvent(type_id, payload);
}
```

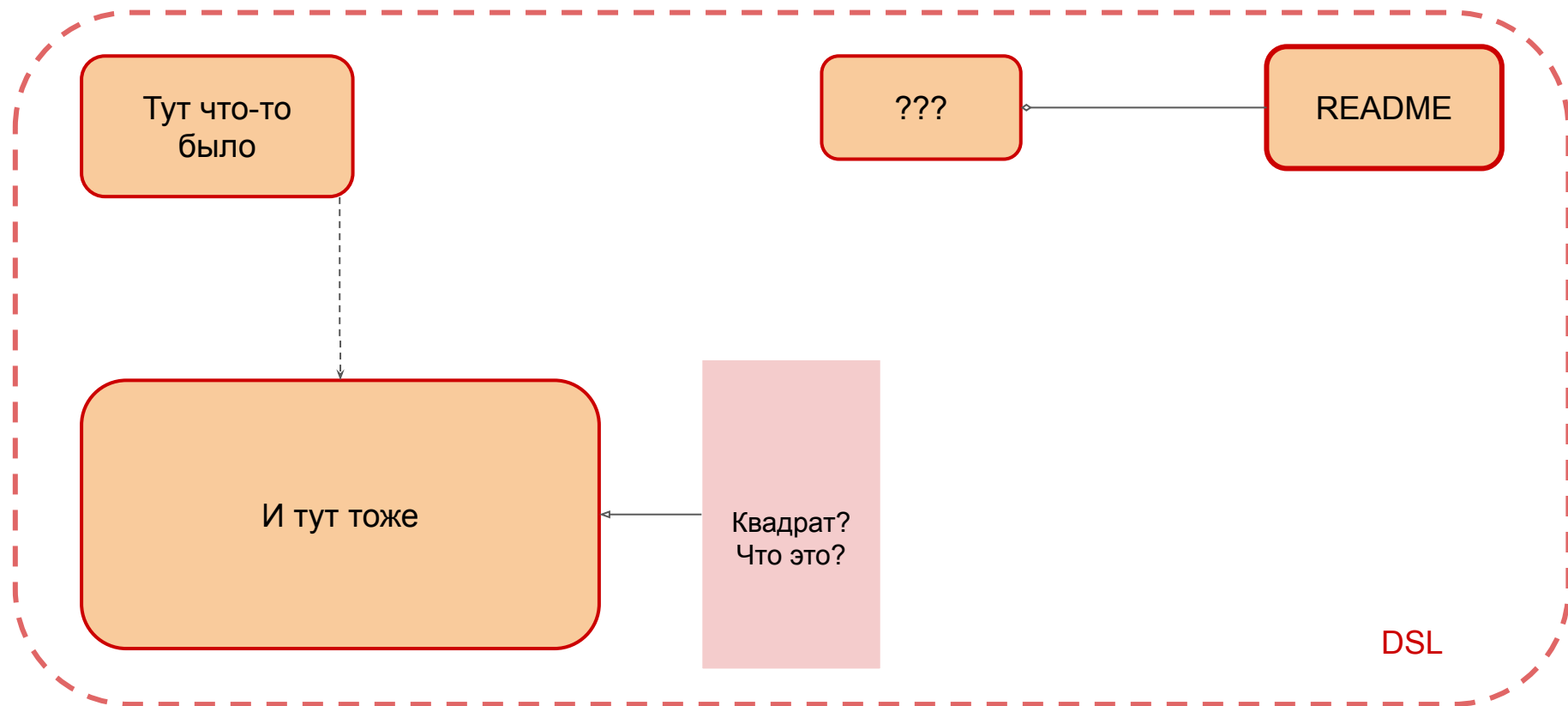
```
format IOp {
    0x0: addi({{
        Rd_sd = rvSext(Rs1_sd + imm);
    }});
    0x2: slti({{
        Rd = (rvSext(Rs1_sd) < imm) ? 1 : 0;
    }});
    0x3: sltiu({{
        Rd = (rvZext(Rs1) < imm) ? 1 : 0;
    }}, uint64_t, imm_code = {{ imm = rvZext(sext<12>(IMM12)); }});
    0x4: xori({{
        Rd = rvSext(Rs1 ^ imm);
    }}, uint64_t);
```

# Домашнее задание 2

# Домашнее задание 2

- Превратите [ProtDSL](#) из песочницы в рабочее решение
- Поддержите rv32IM
- Сгенерируйте симулятор ЦПУ по имеющемуся описанию

# ProtDSL архитектура



# Ссылки

- [What can you do with an ISA specification?](#)
- [Vienna Architecture Description Language](#), [GitHub](#)
- [Pydgin: Generating Fast Instruction Set Simulators](#), [GitHub](#)
- [SAIL](#), [GitHub](#)
- [Simulation Foundations Book, in Russian, Grigory Rechistov](#)
- [The RISC-V Instruction Set, Manual Volume I, Unprivileged Architecture](#)
- [Spike RISC-V ISA Simulator](#)
- [Gem5](#)